

Projekt: Cyfrowy bliźniak i testbench diagnostyczny ładowarek AC

Model-based diagnostyka, weryfikacja licznika i raporty do producenta

Dawid Cekała · zadanie rekrutacyjne AMPERE POINT — część 2 · wersja 3 · 16 czerwca 2026

Streszczenie (w jednym oddechu)

Buduję **cyfrowego bliźniaka** ładowarki AC w Simulink/Simscape: model „idealnej, zgodnej z normą” ładowarki, który służy jako **złoty wzorzec**. Do tego **emulator samochodu** ze scenariuszami i **wstrzykiwaniem usterek**, który prowadzi zarówno wzorzec, jak i (opcjonalnie, przez tani adapter) realną ładowarkę. **Komparator** porównuje zachowania i wystawia werdykt pass/fail oraz **twardy, powtarzalny raport do producenta w Chinach**. Całość projektuję i demonstruję w symulacji — bez kapitału, w ok. 40 h. To zamienia obecny, podstawowy „tester wallboxów”, który firma rozdaje w gratisie, w realne narzędzie diagnostyki i kontroli jakości, dopasowane do modelu „OEM na sterydach”.

1. Najpierw zrozummy ładowarkę AC (od zera)

To moje pierwsze zetknięcie z tymi ładowarkami, więc najpierw układałam fundament — bez tego koncepcja nie ma sensu.

1.1. Ładowarka AC to „inteligentny włącznik”, nie prostownik

Bateria auta przyjmuje prąd stały (DC). Z gniazda płynie prąd przemienny (AC). Zamiana AC→DC (prostownik) siedzi **w samochodzie** (ładowarka pokładowa). Wallbox czy ładowarka przenośna **nie zamieniają prądu** — one tylko bezpiecznie podają AC i **dogadują się z autem**, ile prądu może popłynąć. Wniosek dla projektu: cała „inteligencja” ładowarki AC to **sygnalizacja, sterowanie i zabezpieczenia** — i to właśnie da się modelować i testować od zewnątrz, bez zaglądania do środka.

1.2. Rozmowa ładowarki z autem — Control Pilot (handshake)

Komunikacja idzie jedną żyłą sygnałową — **Control Pilot (CP)**. Ładowarka podaje na nią napięcie, a auto swoimi rezystorami „ściąga” je do konkretnych poziomów; każdy poziom to inny **stan**. Gdy auto jest gotowe, ładowarka nadaje na CP **sygnał PWM** (prostokątny, 1 kHz), którego **wypełnienie** koduje maksymalny prąd. To jest serce normy **IEC 61851** i serce mojego modelu.

Stan	Napięcie CP	Znaczenie
A	+12 V	brak auta (spoczynek)
B	+9 V	auto podłączone, jeszcze nie ładuje
C	+6 V	auto ładuje
D	+3 V	ładowanie z wymuszoną wentylacją (rzadkie)
E / F	0 / -12 V	błąd / ładowarka niedostępna

Kodowanie prądu: **prąd [A] = wypełnienie [%] × 0,6** (w zakresie 10–85%). Np. 25% → 15 A, 50% → 30 A, ok. 53% → 32 A. Druga żyła, **Proximity Pilot (PP)**, mówi rezystorem, jaki kabel jest podłączony i ile prądu wytrzyma.

1.3. Licznik energii — i dlaczego jego dokładność jest krytyczna

Ładowarki AMPERE POINT mają **niezerowalny licznik energii (kWh)**. Ze strony firmy: to filar oferty dla flot — pracownik ładuje auto służbowe w domu, a firma rozlicza koszt na podstawie odczytu licznika (seria Q dodaje historię i eksport w aplikacji Tuya). **Jeśli licznik myli się o kilka procent, klient dostaje złe faktury** — a to spór i utrata zaufania. Dokładność licznika to więc nie detal, tylko realna wartość biznesowa, którą da się i warto testować.

1.4. DLB — równoważenie mocy (gdzie najczęściej coś „nie gra”)

Wallbox PRIME ma **Dynamic Load Balancing (DLB)** do 4 stacji: rozdziela dostępną moc tak, by nie wybić bezpiecznika. To klasyczne miejsce na błędy: ładowarka musi **w porę** (norma: w kilka sekund) zmniejszyć prąd, gdy w domu rośnie inne obciążenie, i ponownie go zwiększyć, gdy auto się odłączy. Wolna albo błędna reakcja = wybity bezpiecznik lub niepotrzebnie wolne ładowanie.

1.5. Mapa awarii — gdzie realnie powstają problemy

- **Handshake CP:** zły poziom stanu, brak diody (auto „udaje” kabel), nieczyste przejścia B→C.
- **Prąd:** ładowarka oferuje złe wypełnienie PWM → za wolno albo ponad bezpiecznik.
- **Reakcja w czasie:** zbyt wolna odpowiedź na polecenie zmniejszenia prądu (DLB).
- **Licznik:** przekłamanie kWh → złe rozliczenia floty.
- **Bezpieczeństwo:** brak reakcji na usterkę (zwarcie CP, brak diody) zamiast odcięcia.
- **Regresje firmware:** nowy firmware od producenta „naprawia jedno, psuje drugie”.

2. Realia AMPERE POINT — i co z nich wynika

Zebrane z ich strony i z rozmowy — to fundament dopasowania projektu:

- **„OEM na sterydach”.** Zamawiają gotowy sprzęt (z Azji), magazynują w Polsce (Tuchom k. Gdańska, wysyłka 48 h), testują, wdrażają i sprzedają bezpośrednio. Hasło ze strony: **„każda jednostka przechodzi wiele testów”**.
- **Zamknięty firmware.** Kod jest producenta (Chiny), nie open source. Firma **nie poprawia kodu** — zgłasza uwagi, a producent przysyła zmodyfikowane elementy lub nowy firmware. Czyli ładowarka to dla nich **czarna skrzynka**.
- **Świadomie BEZ OCPP.** Strona wprost: „bez drogich wdrożeń, bez OCPP, bez skomplikowanych dashboardów”. Stawiają na prosty sprzęt + licznik energii do rozliczeń. Mój projekt **nie może** opierać się na OCPP — działa na warstwie fizycznej (CP/PP), mocy i licznika.
- **Strategia małych prototypów.** Testują niewiele sztuk i szybko adaptują się do klienta, żeby nie topić kapitału. Potrzebują narzędzi **tanich, szybkich i powtarzalnych**.
- **Już rozdają „tester wallboxów” w gratisie.** To podstawowy gadżet (lampki: jest/nie ma napięcia, stan). **Mój projekt to świadome rozwinięcie tego pomysłu** z „świeci się / nie świeci” do realnej diagnostyki i kontroli jakości.

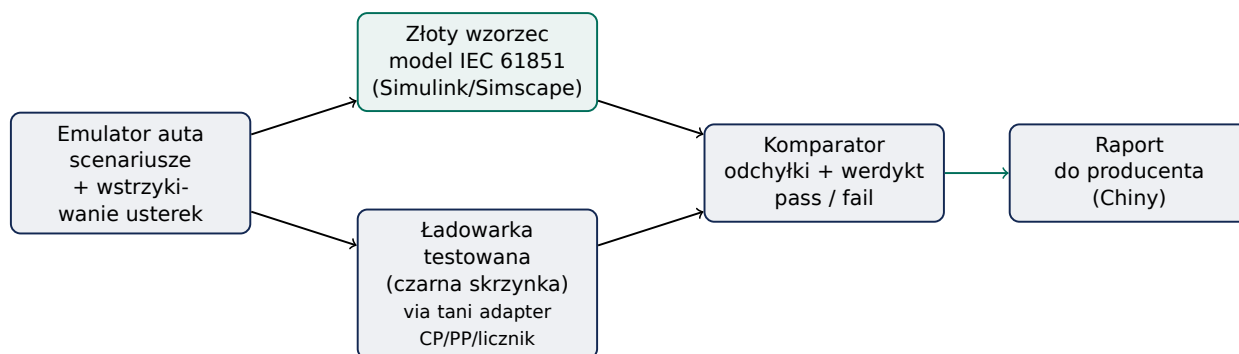
Wniosek: firma potrzebuje taniego, powtarzalnego, automatycznego narzędzia, które (a) charakteryzuje czarną skrzynkę od zewnątrz, (b) łapie regresje po nowym firmware, (c) weryfikuje licznik energii, (d) zamienia mgliste „nie działa” w **twardy raport dla producenta**. Dokładnie to projektuję.

3. Koncepcja: cyfrowy bliźniak + testbench diagnostyczny

Pomysł w jednym zdaniu: skoro nie mam kodu ładowarki, to opisuję jej **poprawne zachowanie** jako model (złoty wzorzec) i sprawdzam, gdzie realna ładowarka od niego odbiega — automatycznie i powtarzalnie.

Cztery komponenty:

1. **Złoty wzorzec (model referencyjny).** Model w Simulink/Stateflow + Simscape, który zachowuje się jak ładowarka **idealnie zgodna z IEC 61851**: stany CP, wypełnienie PWM → prąd, czasy reakcji, logika DLB, licznik. To „prawda”, do której porównujemy.
2. **Emulator auta + scenariusze + wstrzykiwanie usterek.** Druga maszyna stanów udaje samochód i prowadzi test: normalne ładowanie, przypadki brzegowe i celowe usterki (brak diody, złe PP, nagłe zmiany, wolna reakcja, skok obciążenia w DLB).
3. **Komparator + werdykt + raport.** Porównuje odpowiedź wzorca i obiektu testowanego, liczy odchyłki (prąd, czas reakcji, błąd licznika) i wystawia **pass/fail** oraz gotowy **raport do producenta** (co, kiedy, o ile, jak odtworzyć).
4. **Opcjonalny tani adapter sprzętowy.** Mały front-end CP/PP + pomiar, który podcina **realną** ładowarkę do tej samej logiki testowej. Najpierw wszystko projektuję i waliduję w symulacji — sprzęt to cienki most na później.



Czym to się różni od ich obecnego „testera wallboxów”

Obecny gadżet odpowiada na pytanie „**czy w ogóle działa?**” (go/no-go). Mój testbench odpowiada na „**jak dokładnie się zachowuje, gdzie odbiega od normy, o ile, i czy nowy firmware tego nie zepsuł?**” — czyli charakteryzacja, regresja i materiał dowodowy dla producenta.

4. Co dokładnie symuluję (Simulink/Simscape) — i wolne alternatywy

Rdzeń modelu

- **Simscape Electrical — analogowy front-end CP.** Źródło ± 12 V/PWM, sieć rezystorów auta ($2,74\text{ k}\Omega$ + dioda, dołączany $1,3\text{ k}\Omega$), dzielnik. Widzę realne poziomy $12/9/6\text{ V}$ i kształt PWM — dowód, że rozumiem warstwę fizyczną, nie tylko logikę.
- **Stateflow — maszyna stanów CP.** Dwie sztuki: złoty wzorzec (ładowarka) i emulator auta. Pełen handshake $A \rightarrow B \rightarrow C$ i z powrotem, z czasami przejść.
- **Logika prąd/PP.** Przeliczenie wypełnienie $\rightarrow$ prąd ($\times 0,6$), kodowanie PP, wartości „raportowane” jak w aplikacji.

Scenariusz A (kluczowy): weryfikacja licznika energii

Zadaję znany profil prądu w sesji, liczę **prawdziwą energię** (całka mocy po czasie), a model licznika obarczam konfigurowalnym błędem (offset, dryf, kwantyzacja). Testbench wykrywa, gdy **raportowane kWh** odbiega ponad tolerancję (np. $\pm 1\%$). Efekt: zanim sprzęt trafi do floty, wiem, że licznik nie zawyża faktur — a jeśli zawyża, mam dokładną liczbę do producenta.

Scenariusz B (pokaz integracji): DLB

Modeluję dom: bezpiecznik główny + zmienne obciążenie domowe + 1–4 ładowarki z DLB. Podnoszę obciążenie i sprawdzam, czy suma prądów **nigdy nie przekracza** bezpiecznika i czy ładowarka zdążyła zmniejszyć prąd w wymaganym czasie. Odpinam auto i sprawdzam, czy pozostałe „dobierają” zwolnioną moc. To wprost dotyka realnych skarg: wybity bezpiecznik / wolne ładowanie.

Wolne alternatywy (gdyby brak MATLAB-a)

Python (NumPy/SciPy + prosta maszyna stanów, np. transitions) do logiki i scenariuszy; **Scilab/Xcos** jako darmowy odpowiednik Simulinka; **LTspice / ngspice / Falstad** do analogowego obwodu CP. Koncepcja jest niezależna od narzędzia — MATLAB (licencja uczelniana) tylko ją przyspiesza.

5. Plan pracy na ok. 40 h

Faza	Zakres	h
0	Research normy IEC 61851 (CP/PP, czasy) + lista przypadków testowych	4
1	Simscape: analogowy front-end CP (poziomy 12/9/6 V, PWM)	6
2	Stateflow: maszyna stanów — złoty wzorzec + emulator auta	8
3	Logika wypełnienie → prąd + kodowanie PP + wartości raportowane	3
4	Wstrzykiwanie usterek + komparator + werdykt pass/fail	7
5	Scenariusz A: weryfikacja licznika energii (błąd kWh)	5
6	Scenariusz B: DLB (bezpiecznik + obciążenia + 2–4 ładowarki) — elastyczny	4
7	Generator raportu do producenta + demo + opis koncepcji	5
Razem		ok. 42

Twardy rdzeń (fazy 0–5, 7) to ok. 38 h — mieści się w budżecie nawet bez fazy DLB. Faza 6 jest „buforem jakości”: robię, jeśli rdzeń idzie sprawnie.

6. Co to realnie wyłapie (wartość diagnostyczna)

Każdy przypadek to gotowy, odtwarzalny wpis do raportu dla producenta:

- **Licznik zawyża/zaniża kWh** ponad tolerancję → złe rozliczenia floty (uderza w ich główny argument B2B).
- **Złe wypełnienie PWM** → ładowarka oferuje inny prąd, niż powinna (za wolno albo ponad bezpiecznik).
- **Brak reakcji na usterkę** (brak diody, zwarcie CP) zamiast odcięcia → problem bezpieczeństwa.
- **Za wolna reakcja DLB** (> wymóg normy) → ryzyko wybitia bezpiecznika u klienta.

- **Regresja po nowym firmware:** przypadek, który wcześniej przechodził, teraz nie → złapane **przed** wysyłką do klientów.

Zamiast „klient mówi, że coś nie działa” producent dostaje: **który scenariusz, jaka odchyłka, o ile, jak odtworzyć**. To realnie skraca pętlę poprawek z chińską fabryką.

7. Budżet — kapitał bliski zera

Pozycja	Koszt
Symulacja: MATLAB/Simulink/Simscape (licencja uczelniana)	0 zł
albo wariant darmowy: Python + Scilab/Xcos + LTspice	0 zł
Czas: ok. 40 h pracy własnej	wkład własny
<i>Opcjonalnie — dopiero do testów realnej ładowarki:</i>	
Tani adapter CP/PP + pomiar (MCU + analog)	300-500 zł
Ładowarka do testów	sprzęt firmy
Koncept + symulacja (deliverable na poniedziałek)	ok. 0 zł kapitału

To wprost odpowiada na ich strategię „nie topić kapitału”: najpierw tani model, dopiero potem ewentualny sprzęt.

8. Ryzyka i ograniczenia (uczciwie)

- **Model jest tak dobry, jak norma, którą koduję** — dlatego faza 0 (rzetelne wczytanie IEC 61851) jest krytyczna.
- **Czarna skrzynka** — niektóre zachowania wnioskuje z zewnątrz; testbench wskazuje *co* odbiega, nie zawsze *dlaczego* (to już rola producenta).
- **Adapter dotyka napięcia sieci** — dlatego wersja symulacyjna jest pełnoprawnym deliverable, a sprzęt to osobny krok z zabezpieczeniami.
- **To nie certyfikacja** — narzędzie wspiera diagnostykę i QA, nie zastępuje akredytowanych badań bezpieczeństwa.

9. Dlaczego to pasuje do firmy (pitch)

AMPERE POINT zarabia na tym, że **lepiej testuje i szybciej adaptuje** cudzy sprzęt niż konkurencja — „OEM na sterydach”. Mój projekt zamienia ich pętlę „testuj → zbierz feedback → zgłoś producentowi” w **powtarzalny, tani, mierzalny proces**: złoty wzorzec łapie odchyłki, regresja pilnuje nowych firmware’ów, a weryfikacja licznika chroni ich główny argument dla flot. Zero kapitału na start, pełna symulacja jako dowód koncepcji — i jasna ścieżka do wersji sprzętowej, gdy się opłaci.

10. Ścieżka realizacji — od czego zacząć budowę

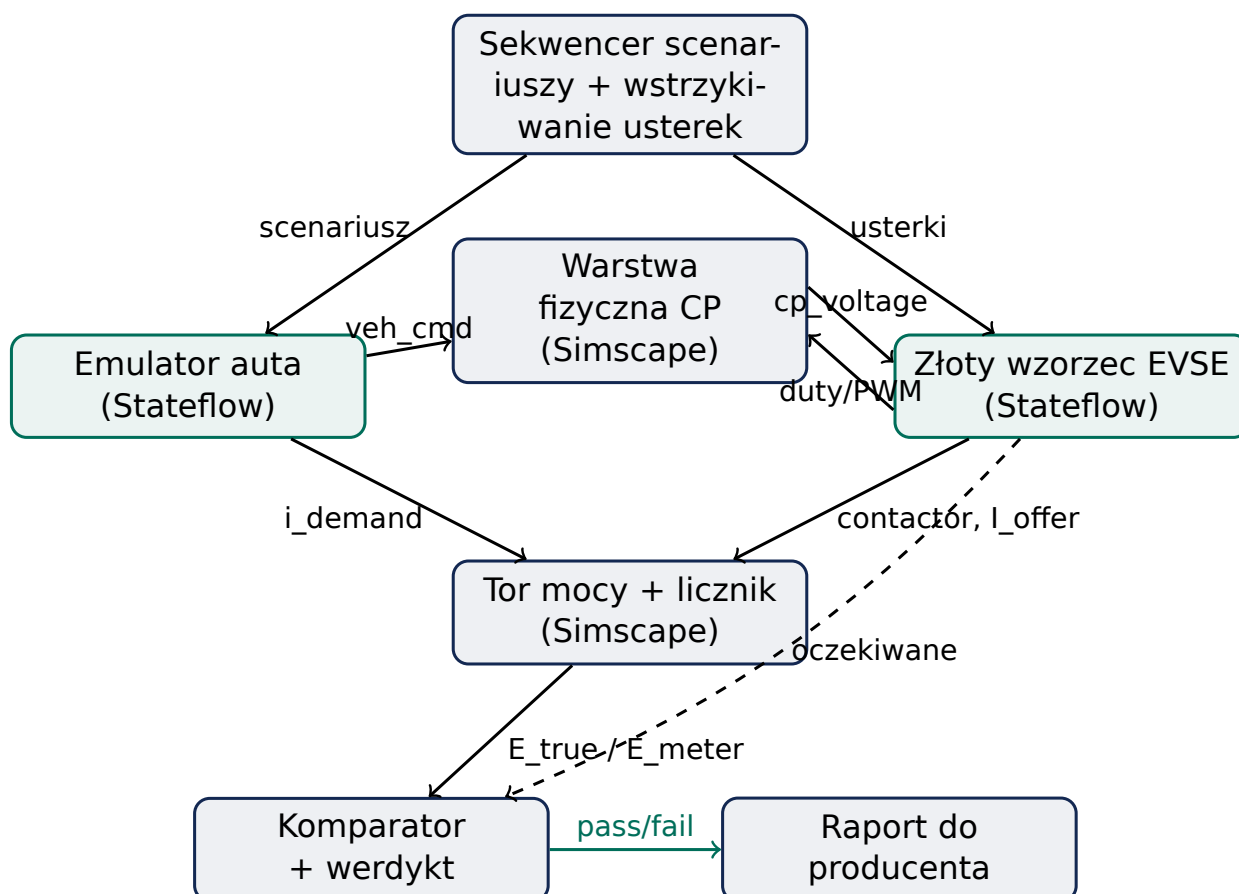
Konkretny szkielet modelu: jakie bloki postawić, jakie stany i sygnały, i **za co dokładnie każdy element odpowiada**. Pisane tak, by dało się usiąść i zacząć — nawet przy pierwszym kontakcie z tymi ładowarkami.

10.1. Trzy rodzaje klocków (orientacja w minutę)

- **Simulink** — diagram sygnałowy. Bloki (Gain, Sum, Integrator, Lookup) przetwarzają *sygnały* (liczby zmienne w czasie); strzałki mają kierunek. Tu żyje matematyka i logika.
- **Simscape Electrical** — symulacja *fizycznego obwodu*. Stawiasz prawdziwe elementy (rezystor, dioda, źródło) i łączysz jak schemat; solver liczy prądy i napięcia z praw Kirchhoffa. Połączenia są dwukierunkowe (fizyczne), nie sygnałowe.
- **Stateflow** — maszyna stanów na Simulinku. Rysujesz stany (prostokąty) i przejścia (strzałki z warunkami); w każdym kroku czasu sprawdza warunki i reaguje. Tu żyje „mózg” ładowarki i auta.
- **Mostki** — Simulink-PS Converter i PS-Simulink Converter przekładają sygnał Simulinka na wielkość fizyczną Simscape’a i odwrotnie. **Solver Configuration** to obowiązkowy blok każdego obwodu Simscape.

10.2. Architektura modelu i sygnały

Sześć podsystemów; strzałki to sygnały płynące między nimi. Emulator auta i wzorzec spotykają się na **warstwie fizycznej CP** (to „kabel”), a osobno na **torze mocy** (to „prąd ładowania”).



Sygnal	Jedn.	Znaczenie	Z → do
cp_voltage	V	napięcie linii Control Pilot	warstwa CP → wzorzec
duty	%	wypełnienie PWM (koduje prąd)	wzorzec → warstwa CP
veh_cmd	—	które rezystory/dioda wpięte	emulator → warstwa CP
I_offer	A	prąd oferowany autu	wzorzec → tor mocy
i_demand	A	prąd, jaki auto chce pobrać	emulator → tor mocy
contactor_cmd	0/1	zamknięcie stycznika mocy	wzorzec → tor mocy
pp_resistance	Ω	kod kabla (Proximity Pilot)	emulator → wzorzec
E_true / E_meter	kWh	energia prawdziwa / zliczona	tor mocy → komparator
verdict	pass/fail	wynik testu + odchyłka	komparator → raport

10.3. Faza 1 — Warstwa fizyczna CP (Simscape Electrical)

Co ma robić: odtworzyć realne napięcia Control Pilot (12 / 9 / 6 V) i sygnał PWM, żebyś widział warstwę fizyczną na wykresie (Scope), nie tylko liczby.

Bloki i ich rola:

- **Controlled Voltage Source** — generuje falę CP; steruje nim sygnał PWM ± 12 V z Simulinka (przez Simulink-PS Converter). To „nadajnik” ładowarki.
- **Pulse Generator** (Simulink) — 1 kHz, wypełnienie ustawiane sygnałem duty. Odpowiada za zakodowanie prądu w szerokości impulsu.
- **Resistor R1 = 1 k Ω** (strona ładowarki, szeregowo) — rezystor pilota; z siecią auta tworzy **dzielnik napięcia** ustalający poziom CP.
- **Resistor R2 = 2,74 k Ω + Diode** (strona auta) — po wpięciu dzielnik daje ok. 9 V = stan B. Dioda jest po to, by ładowarka „widziała”, że to prawdziwe auto (rezystor bez diody = usterka).
- **Resistor R3 = 1,3 k Ω + Ideal Switch** — dołączany równolegle do R2 (przełącznik sterowany przez emulator). Obniża poziom do ok. 6 V = stan C (auto gotowe).
- **Voltage Sensor + PS-Simulink Converter** — mierzy napięcie CP i oddaje je do Simulinka jako cp_voltage. To „oczy” obu maszyn stanów.
- **Solver Configuration + Electrical Reference** (masa = PE) — obowiązkowe dla obwodu.

Skąd te wartości: dzielnik 1 k Ω (ładowarka) z 2,74 k Ω (auto) daje $12 \cdot 2,74 / (1 + 2,74) \approx 9$ V; po dołączeniu 1,3 k Ω równolegle opór auta spada do ok. 0,88 k Ω , a napięcie do ok. 6 V. Stąd progi 9/6 V.

Kamień milowy: na Scope widzisz prostokąt ± 12 V, a poziom górny przeskakuje 9 V $\rightarrow$ 6 V, gdy emulator przełącza stan B $\rightarrow$ C.

10.4. Faza 2 — Maszyny stanów (Stateflow)

Dwie karty: jedna udaje auto, druga to wzorzec ładowarki.

Karta „Emulator auta”

Steruje stroną auta i prowadzi scenariusz. **Stany:** A_Unplugged $\rightarrow$ B_Connected $\rightarrow$ C_Ready (+ Fault). **Wyjścia:** veh_cmd (które rezystory/diodę wpiąć), i_demand (ile prądu auto chce w stanie C). **Wejścia:** komendy ze sekwencera.

Karta „Złoty wzorzec EVSE”

Wzorcowa logika ładowarki wg IEC 61851. **Wejścia:** cp_voltage (czyta poziom → wnioskuje stan auta), pp_resistance, limity prądu. **Stany:** Idle (stałe +12 V, bez PWM) → Detected (wykrył 9 V → włącza PWM o wypełnieniu = $\text{prąd}/0,6$) → Charging (wykrył 6 V → zamyka stycznik) → Fault (nielegalny CP → otwiera stycznik). **Wyjścia:** duty, contactor_cmd. **Wymóg czasowy:** po komendzie zmniejszenia prądu (DLB) wypełnienie zmienia się w ciągu kilku sekund — to później testujemy.

Pętla: wzorzec ustawia duty → Pulse Generator (warstwa CP) → cp_voltage → wraca do obu kart. To zamknięta pętla sterowania, jak w realu.

Kamień milowy: uruchamiasz symulację i widzisz pełny handshake $A \rightarrow B \rightarrow C \rightarrow A$ oraz duty/contactor_cmd reagujące zgodnie z normą.

10.5. Faza 3 — Prąd, PP i wartości raportowane

Co robi: zamienia logikę na konkretne liczby prądu i pilnuje limitów.

- **Gain $\times 0,6$ + Saturation** (6–80 A) — przelicza wypełnienie na prąd i ogranicza do zakresu. (Można 1-D Lookup Table, by ująć regułę 10–85% i wyjątek 5% = komunikacja cyfrowa.)
- **1-D Lookup Table (PP)** — mapuje rezystancję PP na obciążalność kabla (np. $1,5 \text{ k}\Omega \rightarrow 13 \text{ A}$, $680 \Omega \rightarrow 20 \text{ A}$, $220 \Omega \rightarrow 32 \text{ A}$).
- **MinMax** — prąd oferowany = minimum z: limit instalacji, limit kabla (PP), limit DLB. Tak realnie ładowarka wybiera prąd.
- **Podsystem „wartości raportowane”** — liczy, co pokazałby wyświetlacz/aplikacja (prąd, napięcie, energia), by porównać raport z rzeczywistością.

10.6. Faza 4 — Wstrzykiwanie usterek i komparator

Wstrzykiwanie usterek — przełączniki/flagi (panel parametrów lub Signal Builder) celowo psujące model:

- no_diode — wyłącza diodę (wzorzec ma wykryć i przejść w Fault).
- cp_short — wymusza cp_voltage=0 (zwarcie do PE) → stan E.
- duty_error — offset wypełnienia (blok **Sum**); firmware oferuje zły prąd.
- slow_response — **Transport Delay** na zmianie duty; za wolna reakcja na DLB.
- meter_error — **Gain**/offset na liczniku energii.

Komparator — porównuje obiekt z wzorcem i wystawia werdykt:

- prąd oferowany vs oczekiwany: **Sum** → **Abs** → **Compare To Constant** (tolerancja).
- czas reakcji vs próg: timer mierzy odstęp komenda → zmiana duty.
- błąd licznika vs tolerancja; legalność przejść stanów.

Bloki: **Relational Operator**, **Compare To Constant**, **Logical Operator**, (opcjonalnie **Assertion**). Wyjście: verdict + zmierzone odchyłki.

10.7. Faza 5 — Tor mocy i weryfikacja licznika

Co robi: sprawdza, czy zliczona energia (kWh) zgadza się z prawdą — ich filar dla flot.

- **AC Voltage Source** (230/400 V) — sieć.

- **Controlled Current Source** (lub **Variable Resistor**) — udaje ładowarkę pokładową auta pobierającą i_{demand} (do I_{offer}).
- **Current Sensor** + **Voltage Sensor** — mierzą prąd i napięcie.
- **Product** ($P = U \cdot I$) → **Integrator** ($\int P dt$) → **Gain 1/3600** → E_{true} [kWh]. Integrator sumuje pole pod krzywą mocy = energia.
- **Model licznika**: ta sama całka przepuszczona przez `meter_error` (gain/offset/kwantyzacja) → E_{meter} .
- **Komparator**: błąd % = $(E_{\text{meter}} - E_{\text{true}}) / E_{\text{true}} \cdot 100$ → Compare to $\pm 1\%$ → pass/fail.

10.8. Faza 6 — Scenariusz DLB

Co robi: sprawdza, czy ładowarki dzielą moc tak, by nie wybić bezpiecznika.

- **Signal Builder** — profil obciążenia domowego w czasie (rośnie/maleje).
- **1-4 gałęzie ładowarek** — każda to sterowane źródło prądu ustawiane jej I_{offer} .
- **Sum** — prąd całkowity = dom + suma ładowarek.
- **Kontroler DLB** (w karcie wzorca): gdy suma zbliża się do limitu bezpiecznika → zmniejsza duty ładowarek; po odpięciu auta → dobiera wolną moc.
- **Compare To Constant** + **Assertion** — pilnują, że prąd całkowity $\leq$ bezpiecznik *zawsze*; timer mierzy czas reakcji.

10.9. Faza 7 — Sekwencer scenariuszy i raport

- **Test Sequence** (Simulink Test) / karta Stateflow / skrypt MATLAB — uruchamia scenariusze po kolei, ustawia flagi usterek, zbiera werdykty.
- **To Workspace / Simulation Data Inspector** — loguje sygnały.
- **Skrypt raportu** — buduje tabelę: scenariusz, odchyłka, wielkość, jak odtworzyć → eksport do PDF/CSV. To artefakt, który jedzie do producenta.

10.10. Wariant w pełni darmowy (gdyby brak MATLAB-a)

Ta sama koncepcja, mapowanie 1:1:

- **Logika i maszyny stanów** → **Python** (biblioteka `transitions` lub własny FSM; `numpy/scipy` do całek i sygnałów).
- **Obwód CP** → **LTspice** / **ngspice** / **Falstad** (ten sam dzielnik $R1/R2/R3$ + dioda) — weryfikujesz poziomy 9/6 V.
- **Diagram blokowy** → **Scilab/Xcos** (darmowy odpowiednik Simulinka).
- **Raport** → `pandas` → CSV/Markdown/PDF.

10.11. Kolejność budowy i pierwszy kamień milowy

Buduj **od dołu do góry**, mały działający kawałek na raz:

1. Obwód CP w Simscape → zobacz 9 V/6 V (pół dnia). **To Twój pierwszy kamień milowy** — gdy go masz, reszta to nadbudowa.
2. Dołóż dwie karty Stateflow → pełny handshake.
3. Dołóż prąd/PP, potem licznik (faza 5).
4. Dołóż wstrzykiwanie usterek + komparator → pierwszy automatyczny pass/fail.

5. Na końcu DLB oraz sekwencer + raport.

Po każdym kroku masz coś, co działa i co możesz pokazać — a nie wielki model, który „nie wiadomo czemu nie rusza”.